L03 - PyTorch 快速上手

> **"工欲善其事，必先利其器"**

📌 本节目标

1. 理解 PyTorch 是什么以及为什么 MiniMind 使用它
2. 掌握 Tensor（张量）的概念和常用操作
3. 理解自动求导（autograd）和梯度的含义
4. 学会用 `nn.Module` 搭建神经网络
5. 掌握训练循环的完整流程：forward → loss → backward → step

📚 前置知识

- 阅读完 L01 和 L02，对 Transformer 架构有基本印象
- 会用 Python 写简单的代码（变量、函数、循环、列表）
- 如果你完全没学过 Python 也没关系，本节的代码示例都有详细注释

正文讲解

1. 什么是 PyTorch?

PyTorch 是一个开源的深度学习框架，由 Meta（Facebook）开发和维护。它是目前**学术界和工业界最流行**的深度学习工具之一。

> **类比**：如果深度学习是"盖房子"，那 PyTorch 就是你的**工具箱**——里面有锤子（张量运算）、卷尺（自动求导）、电钻（GPU 加速）、蓝图模板（nn.Module）。你不需要从冶铁开始，直接用工具箱里的东西就能搭建复杂的模型。

为什么 MiniMind 选择 PyTorch?

原因	说明
纯原生实现	MiniMind 的核心代码完全用 PyTorch 原生 API 编写，不依赖 HuggingFace 的高层封装
代码可读性	PyTorch 的代码风格接近普通 Python，容易阅读和理解

原因	说明
动态计算图	调试方便，可以随时 print 中间结果
社区生态	与 transformers、vLLM、Ollama 等框架天然兼容
GPU 支持	一行代码切换 CPU/GPU，加速训练

2. Tensor（张量）基础

2.1 什么是张量?

张量是 PyTorch 中**最基本的数据结构**，可以理解为"多维数组"。

> **类比**：张量就是数据的"容器"，不同维度的容器有不同的名字：

维度	数学名称	例子	PyTorch 形状
0 维	**标量 (Scalar)**	温度 `25.5`	`torch.tensor(25.5)` → shape `()`
1 维	**向量 (Vector)**	一个词的嵌入 `[0.1, 0.3, 0.5]`	shape `(3,)`
2 维	**矩阵 (Matrix)**	一批词的嵌入	shape `(4, 768)` → 4 个词，每个 768 维
3 维	**3D 张量**	一个 batch 的句子	shape `(2, 4, 768)` → 2 个句子，每句 4 个词
N 维	**N 维张量**	多头注意力的中间结果	shape `(2, 8, 4, 96)` → batch × heads × seq × dim

在 MiniMind 中，数据几乎全程以张量形式流动。理解张量的形状（shape）变换是读懂源码的关键。

2.2 创建张量

```
import torch

# 从 Python 列表创建
a = torch.tensor([1, 2, 3])           # 一维：向量
b = torch.tensor([[1, 2], [3, 4]])     # 二维：矩阵

# 常用创建函数
zeros = torch.zeros(3, 4)             # 3×4 的全零矩阵
ones = torch.ones(2, 3)               # 2×3 的全一矩阵
rand = torch.randn(2, 768)            # 2×768 的随机正态分布（模拟词嵌入）

# 查看形状
print(rand.shape)                      # torch.Size([2, 768])
print(rand.dtype)                      # torch.float32（默认数据类型）
```

2.3 基本运算

```
x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([4.0, 5.0, 6.0])

# 逐元素运算
print(x + y)       # tensor([5., 7., 9.])
print(x * y)       # tensor([4., 10., 18.])（逐元素乘，不是矩阵乘）
print(x ** 2)      # tensor([1., 4., 9.])

# 聚合运算
print(x.sum())     # tensor(6.)
print(x.mean())    # tensor(2.)
print(x.max())     # tensor(3.)
```

2.4 形状变换（view / reshape）

在 MiniMind 的代码中，你会频繁看到 `.view()` 和 `.reshape()` — 它们都是改变张量形状的操作。

```
x = torch.randn(2, 12)       # 形状 (2, 12)

# 变换为 (2, 3, 4) — 相当于把 12 拆成 3×4
y = x.view(2, 3, 4)
print(y.shape)                # torch.Size([2, 3, 4])

# 用 -1 让 PyTorch 自动推算
z = x.view(2, -1, 4)         # -1 的位置自动推算为 3
print(z.shape)                # torch.Size([2, 3, 4])
```

> **在 MiniMind 中的实际应用**：多头注意力需要把形状为 `(batch, seq_len, d_model)` 的张量变换成 `(batch, n_heads, seq_len, head_dim)`，这就是通过 `view` + `transpose` 完成的。

2.5 矩阵乘法

矩阵乘法是 Transformer 中最频繁的运算（Attention 中的 QK^T、FFN 中的全连接层）。

```
# 二维矩阵乘法
A = torch.randn(3, 4)    # 3×4
B = torch.randn(4, 5)    # 4×5
C = A @ B                # 3×5（@ 是矩阵乘法运算符）
# 等价于 C = torch.matmul(A, B)

# 批量矩阵乘法（Attention 中常用）
Q = torch.randn(2, 8, 10, 96)   # (batch, heads, seq_len, head_dim)
K = torch.randn(2, 8, 10, 96)   # 同上
attn = Q @ K.transpose(-2, -1)  # Q·K^T → (2, 8, 10, 10)
```

3. 自动求导 autograd

3.1 什么是梯度?

> **类比**：假设你蒙着眼睛站在一座山上，想要走到山谷（最低点）。你不能看到全景，但可以感受脚下地面的**倾斜方向**——这就是**梯度**。梯度告诉你"往哪个方向走一小步，下降最快"。

在机器学习中：

- **山** = 损失函数（我们要最小化它）
- **你的位置** = 模型参数的当前值
- **梯度** = 损失对每个参数的偏导数（告诉你如何调整参数才能降低损失）

3.2 PyTorch 的自动求导

PyTorch 最强大的特性之一就是**自动求导**——你只需要定义前向计算，PyTorch 自动帮你算出所有梯度。

```
# 创建一个需要计算梯度的参数
w = torch.tensor(3.0, requires_grad=True)

# 前向计算：y = w^2 + 2w
y = w ** 2 + 2 * w

# 反向传播：自动计算 dy/dw
y.backward()

# dy/dw = 2w + 2 = 2×3 + 2 = 8
print(w.grad)   # tensor(8.)
```

对于 MiniMind 这样有数千万参数的模型，PyTorch 能自动计算损失函数对每一个参数的梯度——这就是深度学习能够工作的基础。

3.3 为什么需要梯度?

训练模型的过程就是不断调整参数，使损失函数变小：

```
当前参数：w = 3.0
梯度：dL/dw = 8.0
学习率：lr = 0.01

更新：w_new = w - lr × 梯度
     w_new = 3.0 - 0.01 × 8.0 = 2.92

重复这个过程几千次，w 就会逐渐趋近最优值。
```

4. nn.Module：搭建神经网络的蓝图

4.1 什么是 nn.Module?

`nn.Module` 是 PyTorch 中**所有神经网络层的基类**。MiniMind 中的每一个组件——Attention、FFN、Block、整个模型——都继承自 `nn.Module`。

```
import torch.nn as nn

class SimpleLayer(nn.Module):
    def __init__(self, input_size, output_size):
        super().__init__()
        self.linear = nn.Linear(input_size, output_size)

    def forward(self, x):
        return self.linear(x)

layer = SimpleLayer(768, 256)
x = torch.randn(2, 768)     # 2 个样本，每个 768 维
output = layer(x)            # 自动调用 forward 方法
print(output.shape)          # torch.Size([2, 256])
```

核心规则：

1. `__init__` 中定义层的**结构**（有哪些参数）
2. `forward` 中定义**前向计算**（数据怎么流过这些参数）
3. 调用时直接用 `layer(x)` 而不是 `layer.forward(x)`

5. 关键网络层详解

5.1 nn.Linear（全连接层 / 线性层）

$y = xW^T + b$

全连接层是最基础的运算，本质就是矩阵乘法加偏置。

```
linear = nn.Linear(768, 256)          # 输入 768 维，输出 256 维
print(linear.weight.shape)            # torch.Size([256, 768])
print(linear.bias.shape)              # torch.Size([256])

x = torch.randn(4, 768)              # 4 个样本
y = linear(x)                        # (4, 768) @ (768, 256)^T → (4, 256)
print(y.shape)                        # torch.Size([4, 256])
```

MiniMind 中的使用场景：

位置	代码
Q/K/V 投影	`self.q_proj = nn.Linear(hidden_size, num_heads * head_dim, bias=False)`
FFN 的 gate/up/down	`self.gate_proj = nn.Linear(hidden_size, intermediate_size, bias=False)`
输出层 lm_head	`self.lm_head = nn.Linear(hidden_size, vocab_size, bias=False)`

注意 MiniMind 中大多数 Linear 层的 `bias=False`，这是现代大模型的常见做法，可以减少参数量。

5.2 nn.Embedding（词嵌入层）

Embedding 把离散的 token ID 映射为连续的向量。

```
embed = nn.Embedding(num_embeddings=6400, embedding_dim=768)
# 6400 = 词表大小 (vocab_size)
# 768  = 嵌入维度 (hidden_size / d_model)

token_ids = torch.tensor([234, 567, 891])   # 3 个 token
vectors = embed(token_ids)                   # (3, 768)
print(vectors.shape)                         # torch.Size([3, 768])
```

> **直觉**：Embedding 就是一个大查找表（lookup table）。里面有 6400 行，每行 768 个数。给一个 token ID（比如 234），就返回第 234 行的那个 768 维向量。

在 MiniMind 中：

```
self.embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)
```

有趣的设计：MiniMind 的 `lm_head`（输出层）和 `embed_tokens` 共享权重——也就是说，输入的"查找表"和输出的"预测层"用的是同一个矩阵。这叫 **Weight Tying（权重共享）**，可以减少参数量并提升效果。

5.3 nn.CrossEntropyLoss（交叉熵损失函数）

这是语言模型训练的**核心损失函数**。

> **直觉**：模型预测"今天天气"后面接什么词时，输出了一个概率分布。如果真实答案是"好"（概率 0.35），那损失函数会惩罚模型——"你应该给'好'更高的概率！"

```
loss_fn = nn.CrossEntropyLoss()

# 模型输出的 logits（未归一化的分数）
logits = torch.tensor([[2.0, 1.0, 0.5, 0.1]])  # 4 个词的分数

# 真实标签（正确答案是第 0 个词）
target = torch.tensor([0])

loss = loss_fn(logits, target)
print(loss)   # tensor(0.4402) — 损失越小越好
```

交叉熵损失的数学公式

$$L = -\sum_{i=1}^{C} y_i \log(p_i) = -\log(p_{\text{correct}})$$

因为 y_i 是 one-hot 编码（只有正确类别为 1，其余为 0），所以简化为：对正确类别的预测概率取负对数。

- 预测概率 = 0.9 → 损失 = -log(0.9) ≈ 0.11（预测得好，损失小）
- 预测概率 = 0.1 → 损失 = -log(0.1) ≈ 2.30（预测得差，损失大）

MiniMind 的训练中，模型对每个位置都预测下一个 token，所有位置的交叉熵损失取平均，就是最终的训练 loss。

6. 优化器：Adam 和 AdamW

6.1 什么是优化器?

优化器决定了**如何利用梯度来更新参数**。

最简单的优化器是 **SGD（随机梯度下降）**：

$w_{\text{new}} = w_{\text{old}} - lr \times \text{gradient}$

但 SGD 有两个问题： 1. 对所有参数使用**相同的学习率** 2. 在"山谷"（损失曲面的狭长区域）中震荡严重

6.2 Adam 优化器

Adam (Adaptive Moment Estimation) 是目前最流行的优化器，它为每个参数自适应调整学习率。

核心思想：

- 维护每个参数的**一阶动量**（梯度的指数移动平均 → 方向信息）
- 维护每个参数的**二阶动量**（梯度平方的指数移动平均 → 步长信息）
- 效果：在平坦的方向走大步，在陡峭的方向走小步

> **类比**：SGD 就像一个新手滑雪者，只会直线冲下去；Adam 就像一个老手，会根据雪坡的坡度和弯度灵活调整速度和方向。

6.3 AdamW：权重衰减版 Adam

AdamW 在 Adam 的基础上加入了**权重衰减（Weight Decay）**——一种正则化技术，防止参数值过大。

$w_{\text{new}} = w_{\text{old}} - lr \times (\text{adam_update} + \lambda \times w_{\text{old}})$

其中 λ 是权重衰减系数。直觉上：**如果一个参数值很大但对损失没太大帮助，就把它往 0 拉一拉**，防止过拟合。

MiniMind 使用 AdamW：

```
optimizer = optim.AdamW(model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
```

Adam vs SGD vs AdamW 对比

特性	SGD	Adam	AdamW
自适应学习率	❌	✅	✅
动量	需手动加	内置	内置

特性	SGD	Adam	AdamW
权重衰减	可选	内置（但实现有 bug）	正确实现
训练效果	泛化好但慢	收敛快	收敛快 + 泛化好
大模型常用	❌	✅	✅✅✅

7. 训练循环：完整流程

一次完整的训练包含以下 4 个步骤，不断循环：

```
model = MyModel()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    for input_ids, labels in dataloader:
        # 第 1 步：前向传播 (Forward)
        logits = model(input_ids)

        # 第 2 步：计算损失 (Loss)
        loss = loss_fn(logits.view(-1, vocab_size), labels.view(-1))

        # 第 3 步：反向传播 (Backward)
        loss.backward()

        # 第 4 步：参数更新 (Step)
        optimizer.step()
        optimizer.zero_grad()   # 清零梯度，为下一步做准备
```

逐步解释

步骤	做了什么	类比
Forward	数据流过模型，计算出预测结果	考试答题
Loss	比较预测和正确答案的差距	老师批改试卷
Backward	计算每个参数对损失的"贡献度"（梯度）	分析哪些知识点没掌握好
Step	根据梯度调整参数	针对薄弱环节重点复习
zero_grad	清除上一步的梯度	清空上次的分析，准备下一次

MiniMind 的训练循环（`trainer/train_pretrain.py`）还包含额外的技巧：

```
# 混合精度训练（节省显存，加速计算）
with autocast_ctx:
    res = model(input_ids, labels=labels)
    loss = res.loss + res.aux_loss
    loss = loss / args.accumulation_steps

# 缩放反向传播
scaler.scale(loss).backward()
```

```
# 梯度裁剪（防止梯度爆炸）
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# 梯度累积
scaler.step(optimizer)
scaler.update()
```

8. 实战：一个最简单的"下一个字符预测"

让我们用 PyTorch 写一个迷你版的语言模型，帮你把上面学到的所有概念串联起来。

```
import torch
import torch.nn as nn

# === 1. 准备数据 ===
text = "hello world hello pytorch hello deep learning"
chars = sorted(set(text))            # 所有不重复的字符
vocab_size = len(chars)               # 词表大小
char_to_id = {c: i for i, c in enumerate(chars)}
id_to_char = {i: c for i, c in enumerate(chars)}

data = torch.tensor([char_to_id[c] for c in text])

# === 2. 定义模型 ===
class TinyLM(nn.Module):
    def __init__(self, vocab_size, hidden_size=32):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.linear1 = nn.Linear(hidden_size, hidden_size)
        self.linear2 = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        h = self.embed(x)               # token_id → 向量
        h = torch.relu(self.linear1(h))  # 非线性变换
        logits = self.linear2(h)         # 向量 → 词表大小的分数
        return logits

# === 3. 训练 ===
model = TinyLM(vocab_size)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(100):
    inputs = data[:-1]     # "hello world ... learnin"
    targets = data[1:]     # "ello world ... learning"

    logits = model(inputs)
    loss = loss_fn(logits, targets)

    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# === 4. 生成 ===
with torch.no_grad():
    seed = char_to_id['h']
```

```
    result = ['h']
    current = torch.tensor([seed])

    for _ in range(20):
        logits = model(current)
        probs = torch.softmax(logits, dim=-1)
        next_id = torch.multinomial(probs, 1).item()
        result.append(id_to_char[next_id])
        current = torch.tensor([next_id])

    print("生成结果:", ''.join(result))
```

这个迷你模型虽然简单（没有 Attention，只有两个全连接层），但它包含了大语言模型训练的**所有核心要素**：Embedding → 前向计算 → 损失函数 → 反向传播 → 参数更新 → 自回归生成。

9. GPU 训练基础

9.1 为什么需要 GPU?

硬件	每秒浮点运算	类比
CPU	~100 GFLOPS	一个非常聪明的人
GPU (RTX 4090)	~83 TFLOPS	一千个普通工人同时干活

GPU 擅长**大量重复的简单计算**（如矩阵乘法），这恰好是深度学习的核心需求。

9.2 PyTorch 中使用 GPU

```
# 检查是否有 GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"使用设备: {device}")

# 把模型和数据移到 GPU
model = model.to(device)
input_ids = input_ids.to(device)
labels = labels.to(device)

# 之后的所有计算都在 GPU 上进行
output = model(input_ids)
```

MiniMind 支持单卡训练，也支持多卡并行训练（DDP）。启动训练时：

```
# 单卡
python trainer/train_pretrain.py

# 多卡 (DDP)
torchrun --nproc_per_node 2 trainer/train_pretrain.py
```

9.3 混合精度训练

MiniMind 使用 **AMP (Automatic Mixed Precision)** 混合精度训练：

- 部分运算用 FP16（半精度），速度更快、显存更省
- 关键运算（如损失计算）保持 FP32（全精度），保证数值稳定

```
scaler = torch.amp.GradScaler()
with torch.autocast(device_type=args.device, dtype=torch.bfloat16):
    output = model(input_ids, labels=labels)
    loss = output.loss
```

📂 对应 MiniMind 源码

概念	对应文件	关键代码
模型定义（nn.Module）	`model/model_minimind.py`	所有 class 都继承 `nn.Module`：`Attention`，`FeedForward`，`MiniMindBlock`，`MiniMindForCausalLM`
nn.Linear（全连接层）	`model/model_minimind.py`	`self.q_proj`，`self.k_proj`，`self.v_proj`，`self.o_proj`，`self.gate_proj` 等
nn.Embedding（词嵌入）	`model/model_minimind.py`	`self.embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)`
CrossEntropyLoss	`model/model_minimind.py`	`loss = F.cross_entropy(logits.view(-1, logits.size(-1)), labels.view(-1))`
优化器 AdamW	`trainer/train_pretrain.py`	`optimizer = optim.AdamW(model.parameters(), lr=...)`
训练循环	`trainer/train_pretrain.py`	`train_epoch()` 函数 — forward → loss → backward → step
混合精度训练	`trainer/train_pretrain.py`	`scaler = GradScaler()` + `autocast` 上下文管理器
GPU 设备管理	`trainer/train_pretrain.py`	`model.to(device)`、`input_ids.to(device)`
梯度裁剪	`trainer/train_pretrain.py`	`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`

🎤 面试考点

Q1: Adam 和 SGD 的区别是什么？为什么大模型通常用 AdamW？

答： - **SGD** 对所有参数使用固定学习率，更新公式为 $w = w - lr \times g$。优点是实现简单、泛化性好；缺点是收敛慢、对学习率敏感、在损失曲面的"峡谷"中震荡。 - **Adam** 为每个参数自适应调整学习率，维护梯度的一阶动量（方向）和二阶动量（步长）。收敛速度远快于 SGD。 - **AdamW** 在 Adam 基础上修正了权重衰减（Weight Decay）的实现。原始 Adam 的 L2 正则化与自适应学习率耦合，效果不理想；AdamW 将权重衰

减解耦，正则化效果更好。 - 大模型通常用 **AdamW** 的原因：(1) 参数量巨大，需要快速收敛；(2) 需要正确的权重衰减防止过拟合；(3) 经过大量实验验证效果最佳。

Q2: CrossEntropy Loss 的公式含义是什么？为什么语言模型用它？

答：交叉熵损失公式为 $L = -\sum_i y_i \log(p_i)$，其中 y_i 是真实标签的 one-hot 编码，p_i 是模型的预测概率。对于语言模型，简化为 $L = -\log(p_{\text{correct}})$，即对正确 token 的预测概率取负对数。

语言模型用它的原因： 1. 语言模型的任务是"预测下一个 token"，本质是多分类问题（从 vocab_size 个候选中选一个），交叉熵是多分类的标准损失函数； 2. 交叉熵等价于最小化模型预测分布与真实分布的 KL 散度，信息论意义明确； 3. 梯度性质好——当预测错误时梯度大，预测正确时梯度小，符合直觉。

Q3: 为什么大模型训练需要梯度裁剪（Gradient Clipping）？

答：深度神经网络在反向传播时，梯度可能会变得非常大（**梯度爆炸**），尤其在序列较长或网络较深时。梯度爆炸会导致参数更新量过大，使模型"跑飞"（损失突然暴增，训练崩溃）。

梯度裁剪（`clip_grad_norm_`）会在梯度的 L2 范数超过阈值时，等比缩放所有梯度使其范数恰好等于阈值。MiniMind 中 `max_norm=1.0`，意味着如果所有梯度的总范数 > 1，就按比例缩小。这保证了参数更新的步长有一个上界，训练更加稳定。

Q4: 什么是混合精度训练？有什么好处？

答：混合精度训练（AMP）在同一个训练过程中同时使用 FP16/BF16（半精度）和 FP32（全精度）。前向传播和大部分反向传播用半精度计算，关键操作（损失函数、参数更新等）用全精度。

好处：(1) 半精度计算速度快约 2 倍（现代 GPU 有专用硬件）；(2) 显存占用减少约一半；(3) 配合 GradScaler 可以保持数值稳定性。MiniMind 使用 `torch.bfloat16`，BF16 的指数位与 FP32 相同，不容易溢出，比 FP16 更稳定。

✅ 自测题

题目 1（选择题）

以下哪个 PyTorch 操作执行的是矩阵乘法？

A. `a * b`
B. `a @ b`
C. `a + b`
D. `a ** b`

> **▼ 查看答案**
>
> **B**。`@` 是 PyTorch 中的矩阵乘法运算符（等价于 `torch.matmul`）。`*` 是逐元素乘法，`+` 是逐元素加法，`**` 是逐元素求幂。

题目 2（选择题）

训练循环中 `optimizer.zero_grad()` 的作用是什么？

A. 将模型参数清零
B. 将梯度清零，为下一次反向传播做准备
C. 清空 GPU 显存
D. 重置学习率

> **▼ 查看答案**
>
> **B**。PyTorch 默认会**累积梯度**（每次 `backward()` 得到的梯度会加到已有梯度上）。所以每次参数更新后需要调用 `zero_grad()` 将梯度清零，否则下一步的梯度会和上一步的梯度混合。注意：梯度累积（Gradient Accumulation）技术就是故意不清零、累积多步梯度后再更新，用于模拟更大的 batch size。

题目 3（简答题）

请解释 `nn.Embedding(6400, 768)` 在 MiniMind 中的作用，以及它和 `nn.Linear` 有什么本质区别。

> **▼ 查看答案**
>
> **作用**：`nn.Embedding(6400, 768)` 创建了一个 6400×768 的查找表。输入一个 token ID（0~6399），它返回对应行的 768 维向量。这是将离散的 token 映射到连续向量空间的过程。 **与 nn.Linear 的区别**： - `nn.Embedding` 的输入是**整数索引**，通过查表操作获取向量，等价于将 one-hot 编码与权重矩阵相乘，但更高效（不需要实际构造 one-hot 向量）； - `nn.Linear` 的输入是**连续向量**，通过矩阵乘法做线性变换； - 数学上，`Embedding(id)` 等价于 `Linear(one_hot(id))`，但实现上 Embedding 是直接查表，计算量为 O(d)，而构造 one-hot 后做矩阵乘法的计算量为 O(V×d)。

🔮 下一节预告

PyTorch 工具箱准备好了，接下来我们将打开 MiniMind 的"引擎盖"——**逐个文件、逐个模块**地探索 MiniMind 项目的结构。你将学会如何 clone 项目、搭建环境、运行第一次推理。

👉 **L04 - MiniMind 项目导览：千里之行，始于配环境**